

Gestor de Atualizações — Documentação Consolidada

Bredas Sistemas · Painel web de gestão de clientes/atualizações + Atualizador Inteligente de ERP (agente local)

Documento único que reúne, atualiza e substitui todos os relatórios, auditorias, especificações e apresentações que existiam soltos em [web/docs/](#). Cada afirmação técnica abaixo foi conferida contra o código-fonte real em setembro de 2026 — onde um documento antigo dizia uma coisa e o código dizia outra, o código venceu, e a divergência está registrada na [seção 5](#).

Versão deste documento	1.0
Data	09 de setembro de 2026
Autor	Antonio Salomão
Substitui	Ver seção 6 — histórico deste documento

Sumário

- 1. Visão geral do projeto
- 2. Painel web — Gestor de Atualizações
- 3. Atualizador Inteligente de ERP — agente local (C#)
- 4. Como verificar
- 5. Auditoria de agosto/set 2026 — o que mudou desde então
- 6. Histórico deste documento

1. Visão geral do projeto

Hoje, atualizar o ERP num cliente da Bredas depende de alguém da equipe entrar remotamente (AnyDesk) no servidor de cada cliente e, na mão: fazer backup do banco, trocar executáveis, rodar scripts de atualização de banco e testar se funcionou — servidor por servidor. Isso consome tempo da equipe de suporte, não escala com o número de clientes, e cada passo manual é uma chance de esquecimento (backup pulado, script no banco errado, cliente interrompido no meio de uma venda).

O projeto ataca esse problema em duas frentes independentes, cada uma num estágio de maturidade diferente:

Frente	O que é	Tecnologia	Estado
Painel web (web/)	Sistema multiusuário para a equipe cadastrar clientes, registrar atualizações/agendamentos, publicar pacotes de versão e acompanhar o parque de agentes instalados	Node.js + Express + SQLite (backend), JavaScript puro sem framework (frontend)	Em produção , uso diário da equipe
Atualizador Inteligente de ERP (atualizador/)	Agente que roda como serviço Windows no servidor de cada cliente: baixa a versão publicada no painel, aplica scripts de banco e distribui os executáveis novos, sem acesso remoto manual	C# / .NET 8 Worker Service, driver Firebird nativo	Pré-piloto — compila, o ciclo completo já rodou de ponta a ponta contra Firebird real, mas ainda falta a Fase 2 (participação do ERP Delphi) para poder atualizar um cliente de verdade

As duas frentes se conectam por uma API HTTP: o painel web publica pacotes de versão por sistema (B_Vendas, B_NFe, B_Ordem, ...); o agente local consulta essa API, baixa o pacote da versão do sistema que ele cuida, e reporta de volta o resultado de cada execução. O painel consolida esses relatos num painel de acompanhamento (quantos clientes estão atualizados, quantos falharam, quantos sumiram).

Arquitetura em três pilares (o agente C#)

Pilar	Tecnologia	Responsabilidade
A Mente — API central	Node.js, Express, SQLite (o painel web da seção 2)	Gerenciar versões, pacotes, clientes, publicação, autenticação web e logs dos agentes
O Músculo — agente local	C# / .NET 8 Worker Service	Operar no servidor do cliente: download, extração, aplicação de scripts SQL, injeção dos executáveis novos
O Rosto — ERP e terminais	Delphi (código existente, b_vendas e afins)	Ler o estado da atualização, avisar o usuário e registrar a autorização no momento certo — esta parte ainda não foi escrita , ver 3.4

2. Painel web — Gestor de Atualizações

2.1 O que é

Reescrita em Node.js + JavaScript puro do app desktop original (Python/Tkinter, pasta **gestor/** na raiz do projeto, mantida intacta como referência). Faz a mesma coisa — controlar atualizações de sistemas em clientes, uma agenda de tarefas internas e o cadastro de clientes — só que agora como um servidor web acessível por várias pessoas ao mesmo tempo, cada uma com seu próprio login.

O que mudou em relação ao app desktop:

- **Login multiusuário.** O app original era de uso individual; agora cada pessoa da equipe tem usuário e senha próprios. A primeira conta criada vira administrador automaticamente; depois disso, qualquer pessoa logada pode convidar novas contas pela tela **Usuários** — só administradores podem remover contas, e ninguém pode remover a própria conta nem a última que resta.
- **Servidor em vez de aplicativo instalado** — um processo Node roda num lugar só (PC, servidor da empresa ou nuvem), e a equipe acessa pelo navegador.
- **Mesmo banco (SQLite), mesmas regras de negócio** — nome de cliente único, renomear cliente propaga o novo nome para o histórico, backup automático a cada início do servidor.
- **Aba Histórico**, nova: registra quem criou/editou/excluiu cada cliente, atualização, agendamento, sistema e conta, e quem restaurou cada backup — necessário agora que várias pessoas usam o mesmo sistema ao mesmo tempo.
- **Aba Sistemas**, nova: filtra clientes por sistema e por uma data de corte opcional, para achar quem ficou pra trás depois de uma mudança grande.
- **Paginação** nas listas que crescem (Atualizações, Agendamentos, Clientes, Histórico), com ordenação calculada no servidor.
- **Segurança de servidor web:** cabeçalhos HTTP de proteção (`helmet`) e limite de tentativas de login por IP — o app original, local e sem login, não precisava de nenhum dos dois.
- **Notificação no Discord** (opcional, `DISCORD_WEBHOOK_URL`): avisa um canal a cada atualização cadastrada e quando um agente do Atualizador automático fica offline/com erro.

Funcionalidades adicionadas em set/2026:

- **Alerta proativo de agente offline/com erro** (`AlertaAgenteService`) — confere sozinho, a cada `ALERTA_AGENTES_INTERVALO_MINUTOS` (padrão 15 min), a situação de cada agente do Atualizador automático e avisa o Discord só na *transição* para "offline" (24h+ sem contato) ou "erro" — não repete o aviso a cada ciclo enquanto o problema continua.
- **Tendência mensal de atualizações** (Resumo) — gráfico dos últimos 12 meses.
- **Grupo/Rede de clientes** — campo opcional para agrupar unidades sob a mesma bandeira.
- **Converter Agendamento em Atualização** — botão que pré-preenche um novo registro de Atualização a partir de uma tarefa de Agendamentos.
- **Tempo médio de resolução por responsável** (Resumo) — só conta tarefas criadas depois desta métrica existir, pra não inventar uma data que não existe.

2.2 Arquitetura do código

```
navegador (client/) <---HTTP/JSON--> Express (server/) <--> SQLite (gestao.db)
```

Um único processo Node serve tudo: os arquivos estáticos do front-end e a API JSON (`/api/...`). Sem build step, bundler ou transpilação em nenhum dos dois lados.

Backend (`server/`), em camadas, de fora para dentro:

```
rotas (routes/) -> controllers/ -> services/ -> database/ (repositórios) -> SQLite
```

- `database/` — uma classe `Database` (conexão, migrações, backup automático) e um `Repository` por tabela. **Só aqui existe SQL** — nenhuma outra camada monta uma query diretamente.
- `services/` — regras de negócio (validação, propagação de rename, cálculo dos indicadores do Resumo, import/export de planilha, login). No Python original ficavam misturadas dentro de cada `View` do Tkinter; aqui viraram classes próprias, testáveis sem simular uma requisição HTTP.
- `controllers/` — finos de propósito: recebem a requisição, chamam o serviço certo, devolvem a resposta.
- `routes/index.js` — só o mapeamento verbo HTTP + caminho → método do controller. Rotas de `/api/auth/...` não passam pelo middleware `requireAuth`; todo o resto passa.
- `Server.js` — classe raiz: cria o `Database`, monta serviços/controllers (injeção de dependência simples, na mão) e configura o Express.

Por que `better-sqlite3` (síncrono) e não `sqlite3` / `node:sqlite`: os métodos de repositório não usam `await`. Foge do padrão assíncrono comum em Node, mas segue a mesma filosofia de código direto que o `sqlite3` do Python já usava — o SQLite lê do disco rápido o bastante para "assíncrono" não trazer benefício, só complexidade.

Erros: `services/errors.js` define `ValidationError` (400) e `NotFoundError` (404) — erros esperados, com mensagem segura de mostrar ao usuário. Qualquer outro erro vira 500 genérico, sem vaziar detalhe interno.

Sessão de login: `database/SqliteSessionStore.js` é uma classe própria (estende `session.Store`) que guarda sessões num `sessions.sqlite` separado, usando a mesma `better-sqlite3` do resto do app — evita depender de `connect-sqlite3`, que traz `sqlite3` + `node-gyp`, cadeia com vulnerabilidades conhecidas de build.

Front-end (`client/`) — JavaScript puro, orientado a objetos, carregado como ES Modules direto pelo navegador, sem bundler:

```
core/App.js -- classe raiz: login vs. shell principal, troca de aba, mantém cada View viva
core/View.js -- classe base: listeners rastreados (removidos no destroy()) + ciclo stale-while-revalid
views/*.js -- uma classe por tela (Resumo, Atualizações, Agendamentos, Clientes, Consultar
               Cliente, Distribuição, Versões, Sistemas, Histórico, Login)
core/*.js -- peças reaproveitadas: SortableTable, Pagination, Autocomplete, Modal, Toast,
               PieChart/BarChart, CommandPalette (Ctrl+K), EmptyState, SwrCache, router, theme
api/ApiClient.js -- único lugar que chama fetch; todo o resto fala com o servidor por ele
```

Cada `View` é instanciada uma única vez (não recriada ao trocar de aba), para não perder o que o usuário estava digitando. Toda vez que a aba fica visível, `App.js` chama `view.refresh()`, que usa **stale-while-revalidate** (`core/SwrCache.js`): o que já foi buscado aparece na hora, a revalidação roda em segundo plano, e a tela só é redesenhada se a resposta for diferente (comparação por serialização estável — `JSON.stringify` puro não serve porque o SQLite não garante ordem de colunas entre consultas). Escrita numa aba invalida o cache das outras que dependem do mesmo dado.

O estado da navegação vive na URL (`#/clientes` , via `core/router.js`): recarregar mantém a tela aberta, e dá para compartilhar o link de uma aba específica.

Toda tela nova deve estender `core/View.js` e usar `this.on(alvo, evento, fn)` em vez de `addEventListener` direto quando o alvo é `document` / `window` — listeners registrados assim são removidos no `destroy()` (sem isso, já causou uma tela fantasma reagindo à tecla `Delete` depois de um novo login).

Por que sem framework: decisão deliberada do projeto — front-end em JavaScript puro, orientado a objetos, sem etapa de build. Cada `View` segue o mesmo papel que tinha em `gestor/views/*.py` no app Tkinter original, só desenhando HTML/CSS em vez de widgets Tkinter.

2.3 Revisão de interface e distribuição — set/2026

Uma revisão ampla do front-end e do módulo de distribuição corrigiu defeitos, introduziu o modelo de "uma versão no ar por sistema" e deu ao painel do Atualizador automático o acompanhamento que faltava.

Defeitos corrigidos

#	Defeito	Onde estava	Correção
1	Página sem <code><h1></code> real	<code>core/App.js</code>	<code><h1></code> que muda por aba, junto com <code>document.title</code>
2	Lista de clientes parados nunca era desenhada — a API devolvia os dados, a tela usava só <code>.length</code>	<code>views/ResumoView.js</code>	Tabela ordenável, fundo tingido conforme o atraso, indicador vira botão que rola até ela
3	Race condition na busca — resposta de <code>"ab"</code> podia chegar depois da de <code>"abc"</code> e sobrescrever a tabela	Todas as telas com busca	<code>ApiClient</code> cancela por chave: requisição nova aborta a anterior de mesma chave
4	Listeners vazando — <code>document.addEventListener("keydown")</code> nunca removido; após expirar sessão, <code>Delete</code> podia excluir por uma tela fantasma	Atualizações, Agendamentos, Clientes	Classe base <code>View</code> com <code>this.on(...)</code> rastreado e <code>destroy()</code>
5	Publicar versão sem <code>try/catch</code> nem trava de botão	<code>views/DistribuicaoView.js</code>	Tratamento de erro, botão travado, mensagem do servidor exibida
6	<code>Escape</code> num campo apagava os 8 campos do formulário, sem volta	Formulários	Limpa e oferece Restaurar num toast
7	<code>Enter</code> amarrado campo a campo, sem <code><form></code>	Formulários	<code><form></code> de verdade com <code>submit</code>

#	Defeito	Onde estava	Correção
8	Exportar ignorava os filtros — filtrar 12 registros e receber um <code>.xlsx</code> com 4.000	<code>AtualizacoesController</code> /repositório	<code>exportAll(search, responsavel)</code> usa as mesmas cláusulas de <code>list()</code>
9	"Último log" de cada agente era <code>MAX(id)</code> , não o mais recente por data	<code>VersaoRepository.agentes()</code>	<code>ROW_NUMBER() OVER (PARTITION BY cnpj ORDER BY criado_em DESC, id DESC)</code>

Exclusão reversível em vez de confirmação: Atualizações e Agendamentos não pedem mais confirmação para excluir — a ação acontece na hora, e um toast oferece **Desfazer** por 7 segundos. Confirmação em toda exclusão vira reflexo (a pessoa clica sem ler); custa um clique a mais sempre e não impede o engano nunca. **Clientes continua pedindo confirmação**, de propósito: um cliente é referenciado pelo nome em todo o histórico, e "desfazer" recriaria o cadastro com id novo — não é o mesmo que nunca ter excluído.

Distribuição: versão por sistema

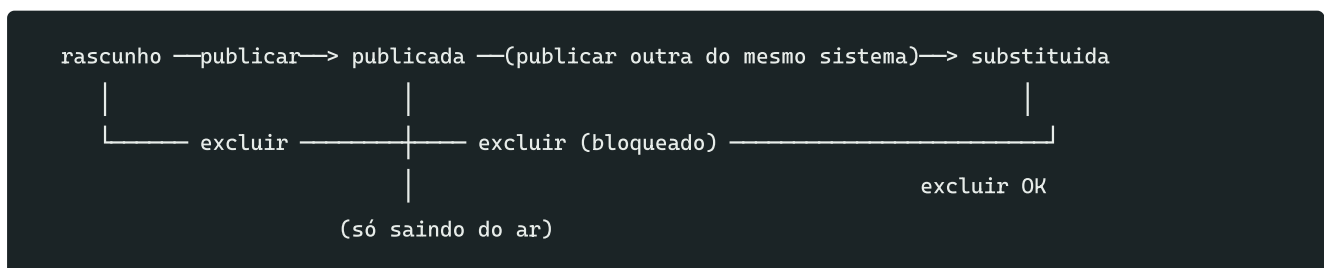
O problema: o formulário só tinha "Versão" e "Arquivo"; `VersaoService.check()` respondia com a última publicada **de qualquer sistema**. Na prática, o agente do B_NFe podia baixar e instalar o pacote do B_Vendas.

O que mudou:

- `sistema` é o primeiro campo do formulário de publicação, obrigatório.
- `check()` exige `sistema` — deixar opcional reabriria o comportamento perigoso por descuido.
- Uma versão no ar por sistema, sempre.** Publicar uma versão nova marca a anterior do mesmo sistema como `substituida`, que sai de circulação na hora. O formulário avisa antes qual versão vai sair do ar.

Por que "substituída" e não excluída: o efeito prático é idêntico (sai do ar imediatamente), mas o registro permanece — quando algo quebra num cliente, a primeira pergunta é "que versão ele estava rodando antes?".

Exclusão de verdade existe (`DELETE /api/versoes/:id`, com o arquivo junto), mas **não é possível excluir a versão que está no ar** — isso deixaria os agentes daquele sistema sem nada para baixar, sem aviso.



Painel do Atualizador automático

Antes, a tabela mostrava contagens calculadas **no navegador**, sobre os últimos 30 registros — "total de execuções" na verdade era "quantos dos últimos 30". E a pergunta central não tinha resposta: **quais clientes ainda não estão na versão que eu publiquei?**

Uma chamada só (`GET /api/versoes/painel`) devolve tudo coerente entre si, com a agregação feita em SQL sobre a tabela inteira:

- **Indicadores gerais:** agentes monitorados · na versão publicada · ainda desatualizados · com erro · sem contato (24h+) · execuções nas 24h.
- **Por agente:** `situacao` (`ok` / `desatualizado` / `erro` / `offline` / `pendente`), última versão instalada vs. versão-alvo publicada, horas sem contato, taxa de sucesso histórica, máquina/hwid/ cidade, contagens reais de sucessos/falhas. Precedência da `situacao` : **offline ganha de tudo** (não dá para afirmar nada sobre uma máquina sumida), e **erro ganha de desatualizado**.
- **Filtros** por situação, sistema e busca livre.
- **Atualiza sozinho a cada 30s**, e **pausa quando a aba não está visível** — um painel de monitoramento com botão manual só mostra a verdade quando alguém lembra de clicar.

Fluidez: cache e renderização

`core/SwrCache.js` + `core/View.js` implementam stale-while-revalidate: mostra o que já tem guardado na hora, revalida por trás, redesenha só se mudou. Redesenhar uma tabela idêntica custa um piscar visível e a perda da posição de rolagem, sem ganho nenhum. Uma barra fina no topo indica revalidação em segundo plano.

Renderização incremental: `SortableTable` reaproveita `<tr>` existentes em vez de reconstruir o `<tbody>` inteiro a cada seleção; `Pagination` só troca rótulos e `disabled` em vez de refazer `innerHTML` (que antes derrubava o foco pro `body` a cada página); `Autocomplete` passou de um listener global por instância (nunca removido) para um único compartilhado com `destroy()`.

Upload com progresso: `postForm` usa `XMLHttpRequest` em vez de `fetch` (só o XHR expõe `upload.onprogress`) — pacotes têm dezenas de MB. Timeout de 10 min para upload, 15s para o resto.

Navegação

Rotas por hash (`#/clientes`) — o hash nunca chega ao servidor, sem risco de conflitar com rota da API. Paleta de comandos (**Ctrl+K**) busca telas, ações e clientes juntos. **Alt+1...Alt+9** vão direto à aba de mesmo número. `?` abre a lista de atalhos. Filtros persistem por aba (`sessionStorage`).

Visual, design system e acessibilidade

CSS consolidado (regras que se sobrescreviam viraram uma definição única por componente); escala tipográfica de 17 tamanhos ad-hoc reduzida a 7 degraus; breakpoints de 6 para 2 estruturais; **tema claro** somado a escuro/sistema (as cores calculadas em JS agora leem os tokens em tempo de execução, em vez de hex copiado que quebraria no tema claro).

Acessibilidade: modal com foco preso e `role="dialog"` (foco padrão em confirmações destrutivas é **Cancelar**, não a ação); abas com `role="tablist"` e navegação por setas; tabelas com cabeçalhos `<button>` (`aria-sort`) e linhas navegáveis por teclado; skip link; `prefers-reduced-motion` zera animações; `aria-live` nos contadores, `role="alert"` nos toasts de erro.

Contrato do agente (Worker C#)

Qualquer agente novo precisa mandar `?sistema=` — é obrigatório desde esta revisão.

```
GET /api/update/check/{cnpj}?sistema=B_Vendas&versao=2026.08.27
x-agent-token: <AGENT_API_TOKEN>
```

```
// há atualização
{ "update_available": true, "sistema": "B_Vendas", "version": "2026.09.01",
  "packages": [{ "file": "...", "url": "...", "sha256": "...", "bytes": 50 }],
  "script_url": "", "notes": "Observações da entrega" }

// já está na última
{ "update_available": false, "sistema": "B_Vendas" }
```

Sem `sistema`: `400 { "error": "Informe o sistema no parâmetro 'sistema'." }`. Um agente que cuida de vários sistemas faz uma chamada por sistema.

```
POST /api/update/log
x-agent-token: <AGENT_API_TOKEN>
```

```
{
  "cnpj": "11222333000181",
  "status": "OK", // obrigatório: OK|SUCESSO|ATUALIZADO|CONCLUIDO ou ERRO|FALHA
  "sistema": "B_Vendas", // sem ele, o painel não sabe contra qual versão alvo comparar
  "versao": "2026.09.01",
  "versaoAnterior": "2026.08.27",
  "duracaoMs": 73500,
  "maquina": "CAIXA-01",
  "hwid": "...",
  "detalhes": "Atualizado e serviço reiniciado"
}
```

Campos novos são opcionais (logs antigos continuam aceitos), mas sem `sistema` / `versao` o agente aparece como "em andamento" em vez de "em dia"/"desatualizado". Aliases em inglês aceitos: `system`, `version`, `previous_version`, `duration_ms`, `machine`, `details`.

Nota de terminologia: o agente C# atual (ver [seção 3](#)) chama esse mesmo valor de `CODIGO_CLIENTE`, não `cnpj` — o campo nunca validou formato de CNPJ de verdade, e o nome do lado do agente foi corrigido para não confundir quem configura um cliente novo. O contrato de rede (parâmetro `cnpj` na URL/JSON) não mudou.

Migrações de banco desta revisão

Todas idempotentes, rodam a cada boot (`Database._migrate`). `versoes_atualizador` ganhou `sistema`, `substituido_em`, `substituido_por`, `tamanho_bytes`; `atualizador_logs` ganhou `sistema`, `versao`, `versao_anterior`, `duracao_ms`, `maquina`; índices `idx_versoes_sistema` / `idx_atualizador_logs_cnpj`. Um backfill automático deduziu o `sistema` das 7 versões publicadas antes desta revisão, comparando o nome do pacote com os sistemas cadastrados — versões cujo sistema não pôde ser deduzido aparecem como "Sistema não informado" e **não são distribuídas** até corrigidas (falhar visível é melhor que falhar calado).

2.4 Segurança e dependências

Todas as dependências de produção foram atualizadas para a major mais recente (set/2026), sem mudança de código além do CSP descrito abaixo:

Pacote	Antes	Depois
<code>express</code>	4.19	5.2
<code>bcryptjs</code>	2.4	3.0
<code>better-sqlite3</code>	11.3	13.0
<code>dotenv</code>	16.4	17.4
<code>helmet</code>	7.1	8.3
<code>multer</code>	2.0	2.3
<code>express-session</code>	1.18	1.19

A migração do `express` fechou a última vulnerabilidade moderada do `npm audit` (`qs`, travada em `~6.15.1` pelo Express 4). Cada rota foi conferida contra as mudanças do Express 5 antes da migração (parser de query string, `path-to-regexp`, assinatura de handler de erro) — nenhuma usava os padrões que mudaram.

O que mudou de fato no código, por causa do endurecimento de CSP feito junto: o script inline de tema no `<head>` de `client/index.html` foi extraído para `client/js/theme-init.js`, e `Server.js` / `requireAgent.js` ganharam uma CSP sob medida e comparação de token em tempo constante.

Deixado de fora de propósito: a vulnerabilidade restante do `npm audit` é em `uuid`, puxada por `exceljs` — não há versão mais nova do `exceljs` que resolva isso sem downgrade de major.

`connect-sqlite3` **foi evitado deliberadamente** desde o início do projeto: depende de `sqlite3` + `node-gyp`, cadeia com vulnerabilidades conhecidas nas ferramentas de build. As sessões usam uma classe própria (`SqliteSessionStore.js`) com a mesma `better-sqlite3` do resto do app.

`.env.bak` esteve commitado no git (corrigido em set/2026): a regra do `.gitignore` só cobria `.env` (nome exato); um `.env.bak` real chegou a ser commitado ("Snapshot antes da migração para Turso", 18/08/2026). O `SESSION_SECRET` dentro era só o valor de exemplo público — nenhum segredo real vazou desta vez, mas o próximo `cp .env .env.bak` de alguém teria vazado credenciais de verdade. Corrigido: `.gitignore` agora ignora `.env.*` (com exceção de `.env.example`), e o arquivo antigo foi tirado do índice do git (`git rm --cached`).

2.5 Como rodar, implantar e operar

Desenvolvimento (Node.js 18+):

```
cd web/server
npm install
Copy-Item .env.example .env
npm run dev
```

Abra `http://localhost:3000`. Na primeira vez, o próprio app mostra uma tela para criar a conta de administrador — não precisa editar arquivo nem rodar comando extra. `npm run dev` reinicia sozinho a cada alteração (via `nodemon`); para produção, `npm start`.

Para reaproveitar o banco do app desktop: copie o `gestao.db` dele para `web/server/data/` (ou aponte `DB_PATH` direto pro arquivo original) — o schema é compatível.

Variáveis de ambiente principais (`server/.env.example` tem a lista completa):

Variável	Para que serve
<code>PORT</code>	Porta do servidor (padrão 3000)
<code>DB_PATH</code>	Caminho do <code>gestao.db</code>
<code>SESSION_SECRET</code>	Assina o cookie de login — trocar por valor aleatório em produção
<code>SESSION_SECURE</code>	<code>true</code> quando atrás de HTTPS
<code>DISCORD_WEBHOOK_URL</code>	Opcional — avisa Discord a cada atualização nova e quando um agente fica offline/com erro
<code>ALERTA_AGENTES_INTERVALO_MINUTOS</code>	Intervalo do alerta proativo (padrão 15)

Backup e restauração: cópia automática do `gestao.db` em `server/data/backups/` a cada início do servidor (mantém as 10 mais recentes). O botão **Backups** no cabeçalho lista e restaura — a página recarrega inteira depois, para garantir que nenhuma tela fique com dado antigo.

Implantação:

- **Rede local:** `npm start` num PC/servidor que fique ligado; a equipe acessa por `http://IP-DA-MÁQUINA:3000`.

- **Pela internet:** proxy reverso (Caddy/Nginx) cuidando do HTTPS na frente, com `SESSION_SECURE=true`. Sem HTTPS, o login trafega sem criptografia.

Como serviço do Windows (recomendado para produção): `Iniciar Gestor.bat` roda numa janela de console em primeiro plano — se ela fechar, o processo travar ou a máquina reiniciar, a equipe fica sem o painel até alguém notar. Para produção, instalar via [NSSM](#):

```
# Num PowerShell como Administrador
cd web
.\instalar-servico.ps1
```

Idempotente (reinstala do zero sem duplicar); baixa o NSSM se preciso, para uma instância manual que já esteja na mesma porta, cria o serviço `GestorAtualizacoes`, com log em `server/logs/` rotacionado por tamanho. `Get-Service GestorAtualizacoes / Restart-Service GestorAtualizacoes / Get-Content server\logs\service-out.log -Tail 50 -Wait`. Para desinstalar: `.\desinstalar-servico.ps1` (não apaga nada do projeto, só o registro do serviço).

Duas pendências conhecidas do serviço (identificadas após instalar de verdade, não impedem o uso):

- **Roda como `LocalSystem`** — o script não define `ObjectName`; o ideal é uma conta virtual por serviço (`NT SERVICE\GestorAtualizacoes`) com permissão NTFS só na pasta do projeto, já que o servidor não precisa de privilégio de SYSTEM para nada do que faz.
- **Log rotacionado mas nunca podado** — `AppRotateBytes` evita um arquivo crescer sem limite, mas as rotações antigas (`service-out-<timestamp>.log`) se acumulam para sempre; o volume é pequeno hoje, mas valeria uma tarefa agendada podando rotações com mais de ~90 dias.

2.6 Limitações conhecidas desta versão

- Só dois níveis de permissão (administrador / usuário comum) — sem papéis mais granulares.
- Sem sincronização em tempo real: cada pessoa vê os dados atualizados ao trocar de aba, não instantaneamente enquanto outra pessoa edita.
- SQLite com `journal_mode=WAL` aguenta bem uma equipe pequena/média; para uso muito intenso e concorrente, a migração natural seria PostgreSQL — não feita nesta versão.

3. Atualizador Inteligente de ERP — agente local (C#)

Estado: pré-piloto. Compila, o fluxo principal está implementado, os bugs críticos conhecidos foram corrigidos, o formato gravado em `BEXE.fdb` foi confirmado campo a campo contra um arquivo real correto, e o ciclo completo (Fase 1 → Fase 3 → Fase 4, com Fase 2 simulada) já rodou de ponta a ponta várias vezes contra Firebird real. **Ainda não deve rodar em cliente real:** falta a Fase 2 (autorização pelo ERP Delphi) e triagem dos scripts antigos aplicados fora do controle da tabela `SCRIPTS`. Detalhe completo em `atualizador/RISCOS-CONHECIDOS.md`.

3.1 Visão geral e objetivo

O agente é um `BackgroundService` (.NET 8) que roda dentro da pasta do cliente, no mesmo servidor onde já ficam `JUNIOR.fdb`, `BEXE.fdb` e os executáveis do ERP. Ele consulta a API central (o painel web da seção 2), baixa e valida os pacotes da versão nova, espera a autorização do usuário (dada pelo próprio ERP Delphi), isola o banco Firebird, aplica os scripts, distribui os executáveis novos e devolve o banco ao ar — sem que ninguém precise entrar remotamente no servidor do cliente.

3.2 Como funciona — máquina de estados

O agente faz polling e reage ao campo `STATUS` da tabela `SYS_ATUALIZACAO`, no `JUNIOR.fdb` do cliente:

Estado	Significado	Quem grava
<code>CONCLUIDO</code> / <code>ERRO</code>	Ocioso — livre para procurar versão nova	Agente
<code>PENDENTE</code>	Pacote baixado, esperando o usuário autorizar	Agente
<code>AUTORIZADO</code>	Usuário confirmou; pode executar	ERP Delphi
<code>PROCESSANDO</code>	Execução crítica em andamento	Agente

Fase 1 — Preparo invisível. Consulta a API com o código do cliente e a versão atual; se houver versão nova, baixa os pacotes, confere o **SHA-256** de cada um, extrai com `7za.exe` e grava `PENDENTE`.

Fase 2 — Decisão do usuário. Não implementada neste repositório. Cabe ao ERP Delphi ler `PENDENTE`, perguntar ao usuário e gravar `AUTORIZADO`. Sem isso o ciclo trava aqui — nos testes registrados, essa fase é simulada gravando `AUTORIZADO` direto no banco via `isql`.

Fase 3 — Execução crítica. `gfix -shut multi -force 0` (isola o banco mantendo acesso SYSDBA) → `gbak` (backup pré) → `ScriptRunnerService` (aplica cada `.sql` pendente do pacote, um processo `isql` isolado por arquivo) → injeção dos binários no `BEXE.fdb` → `gfix -online` → `gbak` (backup pós, já com o banco de volta ao ar).

Fase 4 — Distribuição. Grava os executáveis novos como BLOB na tabela `EXECUTAVEIS` do `BEXE.fdb` (transação única), marca `CONCLUIDO` e reporta à API. Os terminais leem o `BEXE.fdb` e se atualizam sozinhos. Só os `.exe` soltos na **raiz** do pacote entram nessa injeção — dependências em subpastas (ex.: `openssl.exe` usado internamente pelo ERP) ficam de fora, ver 3.8.

Qualquer exceção na Fase 3/4 dispara o `catch`: tenta restaurar o backup pré com `gbak -c -replace_database`, força o banco de volta ao ar, grava `ERRO` com a mensagem em `MENSAGEM_LOG` e reverte `VERSAO_NOVA` para a versão anterior.

3.3 Onde o agente mora

Instalado **dentro da própria pasta do cliente**, numa subpasta própria, para não espalhar `.dll` / `.pdb` / `7za.exe` / backups no meio dos arquivos do cliente:

```
Bredas\                <- pasta do cliente (já existe hoje)
  JUNIOR.fdb
  BEXE.fdb
  B_Vendas.exe, ...
  Atualizador\          <- pasta do agente
    AtualizadorERP.exe
    7za.exe
    atualizador.ini      <- configuração deste cliente (não versionado)
    _trabalho\           <- descartável, recriada a cada ciclo
      pacotes\           <- downloads + extração da versão em andamento
    Backups\             <- PERSISTENTE, nunca apagada pela limpeza automática
      JUNIOR_PRE_9.9.9_20260903_114500.fbk
      JUNIOR_POS_9.9.9_20260903_114500.fbk
```

Por padrão, `JUNIOR.fdb` / `BEXE.fdb` são resolvidos como `..\JUNIOR.FDB` / `..\BEXE.FDB` a partir da pasta do agente (um nível acima). `_trabalho\pacotes\` é a ****única**** pasta que a Fase 4 varre atrás de `.exe` para injetar — qualquer executável que caia ali só é considerado se estiver solto direto nela, não em subpastas. `Backups\` nunca é tocada pela limpeza automática; se poda sozinha, mantendo os últimos `BACKUPS_PARA_MANTER` ciclos (padrão 10).

3.4 Estado atual: pré-piloto

O que já foi validado contra Firebird real, com testes registrados:

- Ciclo completo (Fase 1 → Fase 3 → Fase 4, Fase 2 simulada) rodando **através da API real** (não cópia manual de pacote), pacote publicado real do B_Vendas, como serviço Windows instalado — concluído em 70 segundos.
- Rollback (`gfix -shut` → `gbak -b` → `gbak -c -replace_database` → `gfix -online`) testado de ponta a ponta contra banco real.
- 2302 scripts reais de `Scripts-BVendas/` (2011–2026) testados contra uma cópia de produção de 366 tabelas: 201 pendentes aplicaram limpo; entre 43 e 69 (dependendo do estado prévio do banco) falham por deriva de schema legado e exigem triagem manual — não é bug do agente, é histórico anterior ao controle automático.
- 25 testes de integração automatizados (`AtualizadorERP.Tests/`), contra Firebird real, cobrindo `ProcessService`, `DatabaseService`, `ScriptRunnerService` e o ciclo completo do `Worker`.

O que falta antes de qualquer piloto real:

- **Fase 2 (Delphi) não existe.** Nenhum arquivo `.pas` / `.dpr` no repositório — ler `PENDENTE`, perguntar ao usuário e gravar `AUTORIZADO` ainda precisa ser escrito no ERP.
- **Idempotência dos scripts antigos** — mitigada (o agente nunca reaplica o que já está registrado em `SCRIPTS`, e uma verificação prévia cobre boa parte do que foi aplicado antes de existir esse controle), mas não é 100%: uma fração dos 2302 scripts reais segue precisando de triagem manual.
- **Convenção de `NOMEPRODUTO`** — a injeção usa o nome do arquivo sem extensão; confirmado que funciona para `B_Vendas.exe`, mas não há confirmação de que os terminais realmente leem esse campo (em vez de `NOMEARQUIVO`) para decidir o que baixar.
- Testes automatizados ainda não cobrem a Fase 1 completa contra a API real nem os ~2300 scripts reais de produção (a suíte usa scripts sintéticos pequenos) — vale repetir o ciclo manual contra cópias descartáveis antes de qualquer mudança maior futura no `Worker.cs` / `DatabaseService.cs`.

3.5 Requisitos e instalação

- **.NET 8 SDK** só na máquina que compila — o executável é publicado *self-contained* + *single-file*: todo o runtime .NET e as DLLs de dependência (driver Firebird, `Microsoft.Extensions.*`) ficam embutidos no próprio `AtualizadorERP.exe`. O servidor do cliente **não precisa** ter .NET instalado.
- **Firebird 2.5** no servidor do cliente, com `gfix.exe`, `gbak.exe` e `isql.exe`.
- `7za.exe` ao lado do executável — o pacote gerado pelo CI (GitHub Actions) já inclui essa cópia automaticamente.
- Acesso de leitura/escrita ao `JUNIOR.fdb` e ao `BEXE.fdb`.

Publicar (o mesmo comando que o CI usa a cada push):

```
dotnet build AtualizadorERP.csproj
dotnet publish AtualizadorERP.csproj -c Release -r win-x64 --self-contained true -p:PublishSingleFile=
```

Produce 3 arquivos: `AtualizadorERP.exe` (~70 MB, tudo embutido), `AtualizadorERP.pdb` e `atualizador.ini.example`. Copie `7za.exe` também (dispensável se usar o artefato do GitHub Actions, que já vem completo). Copie a pasta inteira para dentro da pasta do cliente, renomeie `atualizador.ini.example` para `atualizador.ini`, preencha e registre o serviço:

```
sc.exe create "AgenteAtualizadorERP" binPath= "C:\caminho\ate\Bredas\Atualizador\AtualizadorERP.exe" s
sc.exe start "AgenteAtualizadorERP"
```

3.6 Configuração (`atualizador.ini`)

Tudo vem de um `.ini` ao lado do executável — nenhuma credencial embutida no código. Um `.ini` foi escolhido em vez de variável de ambiente porque configurar variável de ambiente de um *serviço Windows* exige elevar e editar o registro — inviável para instalar em dezenas de clientes em campo. Um `.ini` abre no Bloco de Notas.

Chave	Padrão	Obrigatória
<code>CODIGO_CLIENTE</code>	—	sim
<code>SISTEMA</code>	—	sim
<code>API_TOKEN</code>	—	sim
<code>DB_PASSWORD</code>	—	sim
<code>API_URL</code>	<code>http://localhost:3000/api</code>	
<code>DB_USER</code>	<code>SYSDBA</code>	
<code>DB_PORT</code>	<code>3050</code>	
<code>JUNIOR_FDB</code>	<code>..\JUNIOR.FDB</code> (relativo à pasta do agente)	
<code>BEXE_FDB</code>	<code>..\BEXE.FDB</code> (relativo à pasta do agente)	
<code>GFIX_PATH</code> / <code>GBAK_PATH</code> / <code>ISQL_PATH</code>	<code>...\Firebird_2_5\bin\{ferramenta}.exe</code>	
<code>PASTA_TRABALHO</code>	<code>_trabalho</code>	
<code>PASTA_BACKUPS</code>	<code>Backups</code>	
<code>BACKUPS_PARA_MANTER</code>	<code>10</code>	

Faltando qualquer obrigatória, o agente falha ao subir de propósito, para não rodar meio configurado. `atualizador.ini` nunca deve ser commitado (tem credencial real) — já coberto pelo `.gitignore` (`*.ini`); só o `.example`, sem segredo, fica versionado.

`API_TOKEN` precisa bater com `AGENT_API_TOKEN` do servidor. `CODIGO_CLIENTE` não precisa ser CNPJ de verdade — é um identificador livre; recomendado usar o `codigo` já cadastrado na aba **Clientes** do painel (ex.: `C016058`), que o painel casa automaticamente com o cliente. `SISTEMA` precisa bater, letra por letra, com um sistema cadastrado na aba **Sistemas** — uma máquina que roda mais de um sistema precisa de uma instância do serviço por sistema.

A porta `3050` é o padrão do Firebird, mas ambientes reais usam outras — um `BScript.Ini` de produção real inspecionado usava `3051`. Confira antes de instalar num cliente novo.

3.7 Contrato com a API central

Todas as chamadas mandam o header `X-Agent-Token`.

```
GET {API_URL}/update/check/{cnpj}?versao={versaoAtual}
```

```
{
  "update_available": true,
  "version": "2026.08.10",
  "packages": [
    { "file": "pacote.7z", "url": "https://.../api/update/packages/pacote.7z", "sha256": "abc123..." }
  ],
  "script_url": "https://.../BScript.exe"
}
```

`script_url` ainda existe no contrato por compatibilidade, mas **o agente não lê mais esse campo** — a Fase 3 aplica os `.sql` do próprio pacote via `ScriptRunnerService`, não um binário externo (ver 3.8).

```
POST {API_URL}/update/log
```

`{ "cnnpj": "...", "status": "SUCESSO|ERRO", "detalhes": "..." }` — best-effort: falha de rede aqui não interrompe o ciclo.

3.8 Formato gravado em `EXECUTAVEIS ( BEXE.fdb )`

Confirmado campo a campo contra um `BEXE.fdb` real e correto (03/09/2026) — divergir de qualquer um destes formatos faz o atualizador interno dos terminais não reconhecer a linha:

Campo	Formato
<code>NOMEARQUIVO</code>	Caminho completo no disco do cliente (ex.: <code>D:\Bredas\B_Vendas.exe</code>) — a pasta usada é a do <code>BEXE.fdb</code> , não a do agente. Também é a chave de <code>UPDATE</code> vs <code>INSERT</code> .
<code>HASHEXE</code>	SHA-1 em hexadecimal maiúsculo (40 caracteres) — não SHA-256.
<code>VERSAO</code>	A versão embutida no próprio executável (<code>FileVersion</code> , ex.: <code>26.9.1.8</code>), não a versão do pacote publicada no painel.
<code>VERSAOATUALIZADA</code>	Um flag de texto (<code>"True"</code> / <code>"False"</code>), não uma versão — a coluna real só cabe 5 caracteres. O agente só grava <code>"True"</code> ; reverter para <code>"False"</code> é responsabilidade de outra parte do sistema, fora deste repositório.
<code>EXECUTAVEL</code>	BLOB com o conteúdo binário completo do <code>.exe</code> .
<code>DATA_ATUALIZACAO</code>	Data (sem hora) da injeção.

3.9 Histórico de correções críticas

Linha do tempo resumida dos achados de maior impacto, a partir de leitura de código, engenharia reversa dos binários reais (`BScript.exe`, `BEXE.FDB`, `BScript.Ini`) e dos primeiros testes de ponta a ponta. Detalhe completo, com trecho de código e cenário de falha de cada item, em `atualizador/RISCOS-CONHECIDOS.md`.

Data	Achado	Correção
28/08	BScript.exe baixado via script_url era distribuído aos terminais como se fosse atualização do ERP	TEMP_PATH dividido: só pacotes\ é varrida pela Fase 4
28/08	Rollback podia restaurar um backup de dias atrás (arquivo antigo não apagado)	preBkp apagado no início do ciclo; flag backupValido só liga após gbak ter sucesso
31/08	BScript.exe real não tem modo silencioso — confirmado por dois testes reais: /silent é ignorado, a janela do Delphi/FireDAC abre e trava esperando clique	Substituído por Services/ScriptRunnerService.cs : aplica cada .sql via isql isolado por processo, reaproveitando a tabela SCRIPTS que o BScript.exe já mantinha (compatível com o histórico gravado manualmente)
31/08	Backoff quase nunca entrava em ação (contador zerava antes de crescer)	Só zera quando o ciclo anterior estava CONCLUIDO , não em qualquer sucesso de polling
31/08	"Sucesso" registrado mesmo sem nenhum .exe extraído	InjetarNovosBinarios lança erro se a lista vier vazia
31/08	Reversão de VERSAO_NOVA dependia de um .txt solto que podia sumir	Nova coluna VERSAO_ATUAL , separada de VERSAO_NOVA — só avança após sucesso real; nada para "reverter" se falhar antes
31/08	Senha do Firebird exposta na linha de comando (visível via Gerenciador de Tarefas/WMI)	Credenciais via ISC_USER / ISC_PASSWORD no ambiente do processo, não mais como argumento
31/08	API fora do ar (401, DNS morto) contava como "sem atualização" e zerava o backoff	Exceção sobe até o catch do Worker , entra no mesmo backoff de falha de atualização
31/08	Backup pós-atualização rodava dentro da janela de shutdown, somando até 15 min de indisponibilidade	Roda depois do gfix -online , com o banco já servindo os terminais
31/08	Downloads com timeout fixo de 100s (padrão do .NET)	HttpClient sem timeout fixo; cancelamento via CancellationToken propagado
31/08	-shut force_0 nunca foi sintaxe válida do gfix (testado: erro "Target shutdown mode is invalid")	-shut multi -force 0 — "multi" isola terminais mantendo acesso SYSDBA (testado que "full" bloqueia até o SYSDBA)
31/08	gfix / gbak com caminho puro podiam resolver para a instância errada do Firebird (máquina com 2.0 e 2.5 instalados)	Todas as chamadas usam localhost/{porta}:{caminho} explícito
31/08	Connection pooling do driver .NET quebrava depois de um gfix -shut (conexão em cache ficava inválida)	Pooling=false na connection string
01/09	SYS_ATUALIZACAO não existe no JUNIOR.fdb real de produção (366 tabelas inspecionadas) — era só assumida pelo projeto	GarantirTabelaSysAtualizacao cria a tabela no primeiro ciclo se não existir (idempotente)
03/09	openssl.exe (dependência interna do B_Vendas, numa subpasta do pacote) era injetado como se fosse um produto novo	Varredura da Fase 4 restrita a .exe soltos na raiz do pacote

Data	Achado	Correção
03/09	4 dos 6 campos de EXECUTAVEIS gravados em formato errado (nome sem caminho completo, SHA-256 em vez de SHA-1, versão do pacote em vez do FileVersion , etc.) — achado comparando campo a campo contra um BEXE.fdb real e correto	InjetarNovosBinarios reescrito contra o formato confirmado (ver 3.8)
03/09	Configurar o serviço via variável de ambiente exigia elevar e editar o registro do Windows — inviável para instalar em campo	Configuração inteira migrada para atualizador.ini (ver 3.6)
03/09	Backups pré/pós eram apagados no mesmo ciclo em que nasciam (Directory.Delete da pasta de trabalho)	Worker.ArquivarBackups move os dois para PASTA_BACKUPS , fora da limpeza automática; poda mantém as últimas 10 gerações

Todas as correções acima passam pelo **ProcessService**, que exige timeout obrigatório em toda chamada a processo externo — não é estilo, é segurança: a Fase 3 roda com o banco em **-shut force_0 / multi -force 0** (bloqueado para todos os usuários), então um processo que trava sem timeout deixaria o cliente inteiro parado até alguém perceber.

4. Como verificar

Painel web:

```
cd web/server && npm start
```

O console mostra a migração na primeira vez.

O quê	Como
Rota na URL	Trocar de aba → URL vira #/clientes ; recarregar mantém a tela
Paleta	Ctrl+K , digitar o nome de um cliente, Enter → abre a ficha
Tema	Botão no cabeçalho cicla sistema → escuro → claro; recarregar mantém
Cache	Ir e voltar entre abas → aparece na hora, com a barra fina revalidando
Resumo	Clicar em "Parados há mais de 60 dias" → rola até a lista
Distribuição	Escolher sistema → aviso diz qual versão sai do ar
Substituição	Publicar → toast diz qual saiu; a antiga vira "Substituída"
Trava de exclusão	Tentar excluir a versão "No ar" → recusa explicada

O quê	Como
Upload	Enviar pacote grande → barra de progresso
Desfazer	Excluir uma atualização → toast com "Desfazer"
Escape	Preencher o formulário, Esc → toast com "Restaurar"
Exportar	Filtrar e exportar → atualizacoes-filtrado.xlsx só com o filtrado
Teclado	Tab até a tabela, ↑ / ↓ navega, Enter seleciona
Atalhos	? abre a lista

```
# sintaxe de todos os arquivos
cd web && for f in $(find client/js -name '*.js'); do node --input-type=module --check < "$f"; done
for f in $(find server/src -name '*.js'); do node --check "$f"; done
```

Agente C#:

```
cd atualizador
dotnet build AtualizadorERP.csproj
dotnet test AtualizadorERP.Tests
```

Os 25 testes de integração rodam contra Firebird real (não mockado) — cobrem **ProcessService**, **DatabaseService**, **ScriptRunnerService** e o ciclo completo do **Worker**, incluindo o formato de **EXECUTAVEIS** e a retenção de backups. Para validar o ciclo ponta a ponta contra um cliente de teste, ver o passo a passo em **atualizador/README.md** (Fase 2 precisa ser simulada gravando **AUTORIZADO** direto no banco via **isql**, já que ainda não existe no ERP Delphi).

5. Auditoria de agosto/set 2026 — o que mudou desde então

Em 27/08/2026, uma auditoria técnica comparou três documentos de arquitetura anteriores — **Planejamento_Tecnico_Atualizador_ERP_v4** (especificação técnica completa, propondo MD5 e um ciclo de 4 estados sem **PROCESSANDO**), uma apresentação executiva com a mesma arquitetura em linguagem de negócio, e um "Documento Técnico v1.0" já revisado (SHA-256, 5 estados incluindo **PROCESSANDO**) — contra o código-fonte real e, numa segunda passada, contra os próprios binários e dados de produção (**BScript.exe**, **BEXE.FDB** de 44 MB, **BScript.Ini**, 1026 scripts DDL reais).

O resultado da auditoria: 14 comportamentos de segurança/orquestração confirmados batendo com o documento técnico v1.0, e 12 achados novos (2 críticos, 4 de alta severidade, 3 de média, 3 de baixa) que nenhum dos três documentos de arquitetura tinha como revelar — viviam em decisões de implementação só visíveis lendo o C#/Node.js linha a linha, ou abrindo os próprios binários e a base de produção.

Todos os achados dessa auditoria foram corrigidos nas semanas seguintes — a tabela completa, com data de cada correção, está na [seção 3.9](#). Os dois achados críticos da auditoria, em particular, motivaram as duas maiores mudanças de desenho do projeto:

Achado crítico da auditoria (27/08)	O que virou
<code>BScript.exe</code> real parece ser um programa gráfico interativo, sem nenhum modo silencioso — automatizá-lo podia travar o servidor do cliente inteiro	Confirmado por teste real em 31/08 (<code>/silent</code> ignorado duas vezes) → <code>BScript.exe</code> substituído por <code>ScriptRunnerService</code> , que aplica os <code>.sql</code> diretamente via <code>isql</code>
Uma atualização que falha some do radar: <code>VERSAO_NOVA</code> era gravado antes da autorização e nunca revertido em falha, então o próximo polling via "nada de novo"	Corrigido em 31/08 com a coluna <code>VERSAO_ATUAL</code> , separada do alvo em disputa

Os achados de alta/média severidade também foram todos endereçados: o schema real do `BEXE.fdb` (tabela `EXECUTAVEIS`, não `VERSOES_EXE`) foi confirmado e, depois, corrigido campo a campo (achado novo em 03/09, quando uma comparação mais profunda revelou que 4 dos 6 campos ainda estavam no formato errado); a porta do Firebird virou configurável; as credenciais deixaram de ir na linha de comando; a injeção do BEXE ganhou verificação de lista vazia; o backoff foi corrigido para valer mesmo depois de uma falha real de atualização, não só de download.

Um relatório de revisão técnica complementar, também de 27/08, comparou os mesmos três documentos contra um desenho anterior descartado (pipeline de 25 etapas com rename de banco via `n8n`) e recomendou tratar o projeto atual como a direção definitiva — recomendação que se confirmou: a técnica de isolamento via `gfix -shut` (em vez de parar o serviço Firebird inteiro ou fazer rename/cópia do banco) segue sendo a base do ciclo até hoje.

O que ficou sem solução de código, porque depende de decisão/trabalho fora deste repositório: a Fase 2 em Delphi (nenhum `.pas` / `.dpr` existe), o schema real de `JUNIOR.fdb` além de `SYS_ATUALIZACAO` (o agente cria essa tabela sozinha, mas o resto do banco nunca foi auditado por falta de cópia disponível), e a idempotência definitiva dos ~2300 scripts históricos — todos listados como pendências ativas na [seção 3.4](#).

6. Histórico deste documento — o que foi consolidado

Este documento substitui os seguintes arquivos, que existiam separadamente em `web/docs/` e foram removidos após terem seu conteúdo relevante incorporado acima (o histórico completo de cada um continua disponível no `git log`, se for preciso consultar o texto original):

Arquivo removido	Natureza	Para onde foi
<code>ARCHITECTURE.md</code>	Arquitetura do código do painel web	Seção 2.2

Arquivo removido	Natureza	Para onde foi
RELATORIO_IMPLEMENTACAO.md	Relatório de implementação do agente C#, 26–27/08	Superado pelo estado real de 03/09 — seção 3 e seção 5
REVISAO_INTERFACE.md	Revisão de interface/distribuição do painel, set/2026	Seção 2.3
Planejamento_Tecnico_Atualizador_ERP_v4.pdf	Especificação técnica original (MD5, sem estado PROCESSANDO)	Contexto histórico na seção 5
Projeto_Arquitetura_Atualizador_ERP.pdf	Apresentação executiva da mesma arquitetura	Contexto histórico na seção 5
Documento_Tecnico_Atualizador_ERP.html / .pdf	Documento técnico v1.0 (26/08)	Substituído pela seção 3 , que reflete o estado atual
documento-oficial-atualizador-erp.pdf / .docx	Auditoria técnica completa (27/08)	Resumida na seção 5 ; achados individuais na tabela da seção 3.9
auditoria_atualizador_erp.html	Mesma auditoria, versão HTML estilizada	Idem acima
relatorio-atualizador-erp.docx	Revisão técnica complementar (27/08), comparando contra um desenho anterior descartado	Resumida no último parágrafo da seção 5
apresentacao-atualizador-erp.docx	Proposta de projeto em linguagem executiva	Conteúdo condensado na seção 1
Apresentacao_Atualizador_Inteligente_ERP.pptx	Slides gerados a partir do documento técnico e das capturas de tela	Conteúdo coberto pelas seções 1–3; os slides em si não têm informação que não esteja aqui
tela-distribuicao.png , tela-resumo.png	Capturas de tela usadas nos slides acima	Removidas junto com a apresentação — ilustravam a mesma interface descrita na seção 2
gerar-apresentacao.js , gerar-pdf-texto.js	Scripts geradores dos artefatos acima	Sem função depois que os artefatos que geravam deixaram de existir

Por que consolidar: os documentos acima descreviam o mesmo projeto em estágios sucessivos — uma proposta inicial, três especificações técnicas concorrentes, uma auditoria comparando-as com o código, e um relatório de correções — cada um congelado na data em que foi escrito. Entre o relatório de implementação (27/08) e a revisão mais recente do agente (03/09), o código avançou o suficiente para tornar boa parte do conteúdo desatualizado (o **BScript.exe** que o relatório descrevia como corrigido com timeout foi, na verdade,

substituído por completo dias depois). Manter nove documentos derivados uns dos outros, sem um único dono, é como a divergência entre "Documento Técnico v1.0" e o código real aconteceu em primeiro lugar. Este arquivo é, a partir de 09/09/2026, a única fonte para este assunto em [web/docs/](#).